TC5061 · Actividad 3 — Código de experimentación base con MLflow

Alumno: Victor Yoguel Salazar Alanis · A01797950 **Curso:** TC5061 – Operaciones de Aprendizaje Automático (MLOps) · Tecnológico de Monterrey **Repositorio:** https://github.com/CFOP2357/Wine-Quality

---

1. Ejecución del script

El notebook de la Actividad 2 se migró al paquete `wine_quality/` (carga y partición en `dataset.py`, preprocesamiento en `features.py`, selección de modelo en `modeling/models.py` y entrenamiento en `modeling/train.py`); el notebook se conserva solo para el EDA. El script corre sin argumentos (defaults: regresión lineal, semilla 42) y expone por `argparse` el modelo, la semilla, cinco hiperparámetros de modelo, el preprocesamiento (imputación, normalización, escalado) y el early stopping:

```
python -m wine_quality.modeling.train
python -m wine_quality.modeling.train --model rf --n-estimators 300 --max-depth 12 --normalizer none --scaler none
```

También es ejecutable como proyecto MLflow: `mlflow run . --env-manager local -P model=rf -P n_estimators=300`. Las instrucciones completas (instalación, datos, tabla de argumentos) están en el README del repositorio.

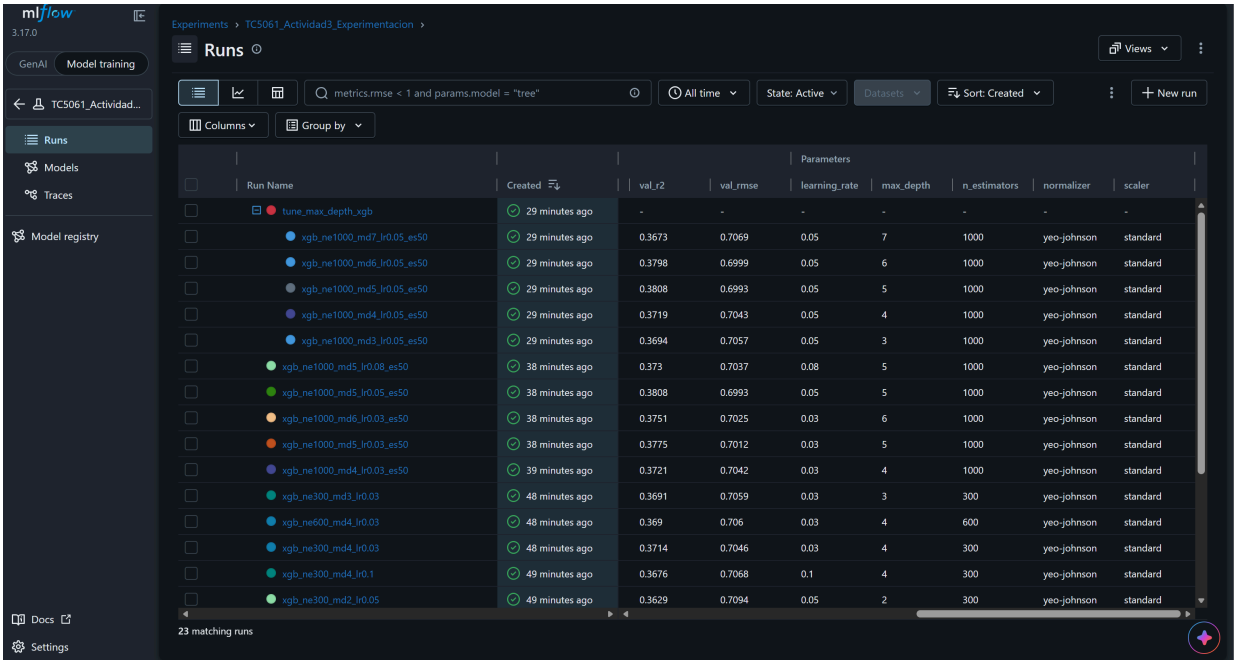
2. Experimento registrado

Todas las corridas se agrupan en el experimento **TC5061_Actividad3_Experimentacion** (tracking local en SQLite). Cada corrida registra, dentro de un `mlflow.start_run()`: los hiperparámetros efectivos del modelo (`log_params`), 16 métricas (`log_metric`: MAE, RMSE y R^2 en train/val/test, la referencia dummy y la brecha de overfitting), artefactos (`log_artifact`/`log_table`: figura real-vs-predicho, tabla de métricas y, con early stopping, la curva train-vs-val por iteración) y el pipeline entrenado completo (`mlflow.sklearn.log_model`). El experimento contiene **23 corridas**: 7 corridas base comparando los cuatro modelos (linear, árbol, random forest, XGBoost) y variando su preprocesamiento, más la búsqueda de hiperparámetros de XGBoost (descenso por coordenadas, early stopping con `eval_set` y un sweep de `max_depth` con corridas anidadas).

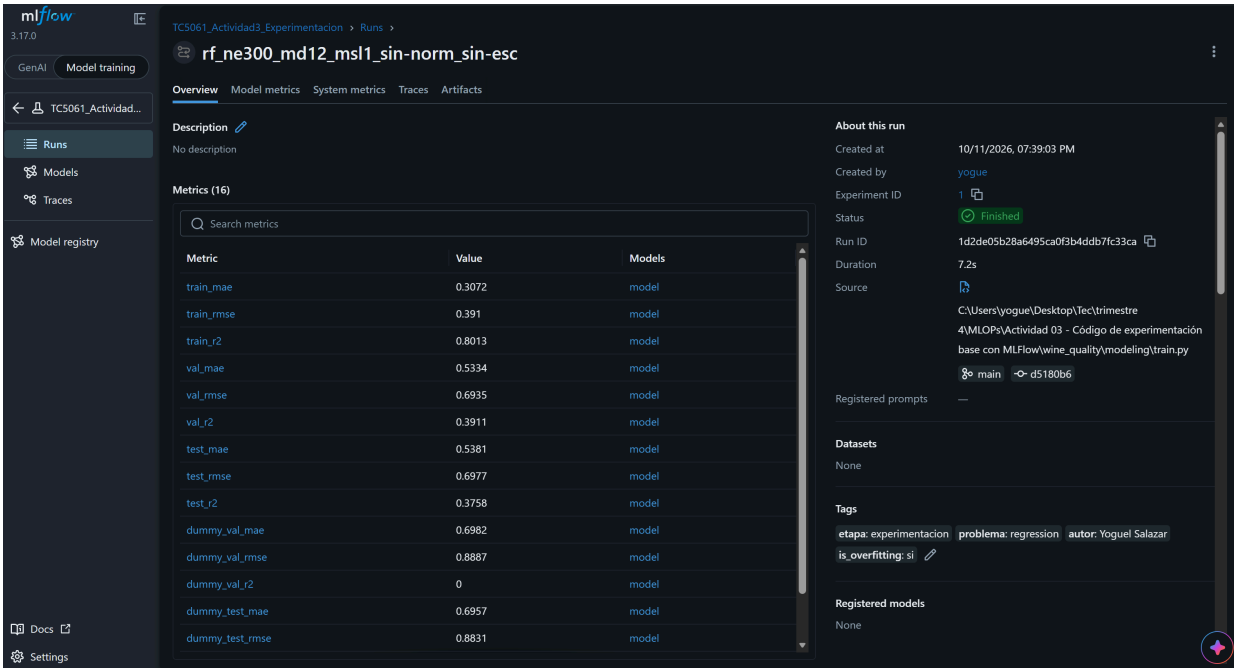
Protocolo: los modelos se compararon y seleccionaron únicamente con las métricas de **validación**; el RMSE de **test** se reporta como estimación de generalización de la configuración ya elegida y no participó en ninguna selección.

3. Evidencias (capturas de MLflow)

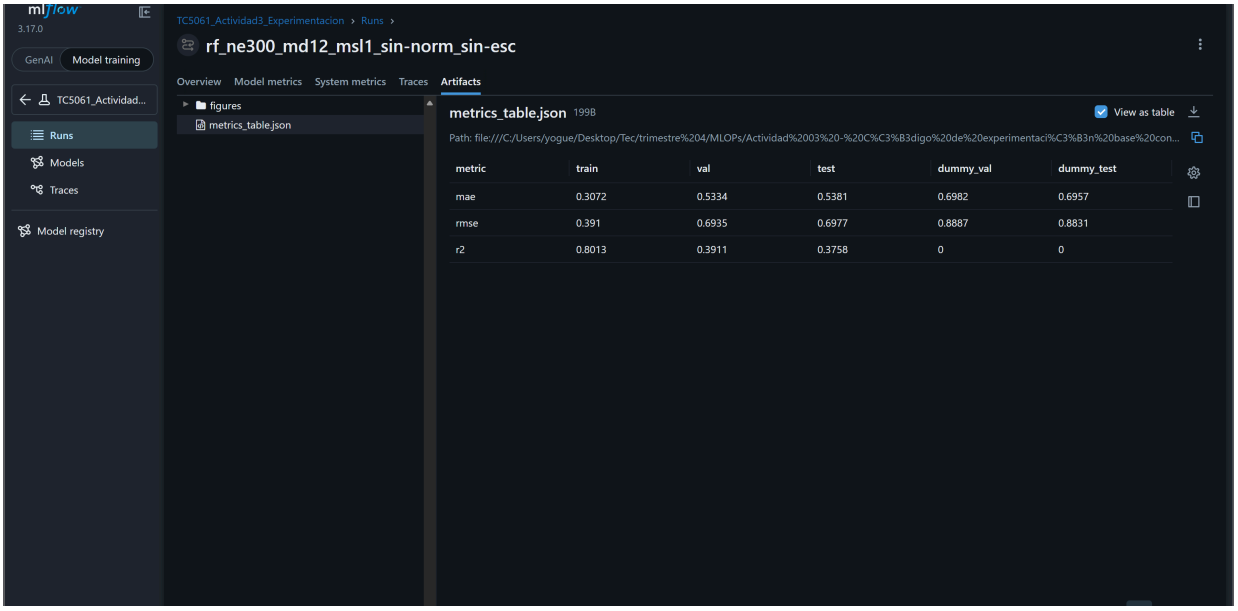
Captura 1 — Tabla comparativa de corridas (23 corridas del experimento, con hiperparámetros y métricas de validación; las hijas del sweep aparecen anidadas bajo `tune_max_depth_xgb`):



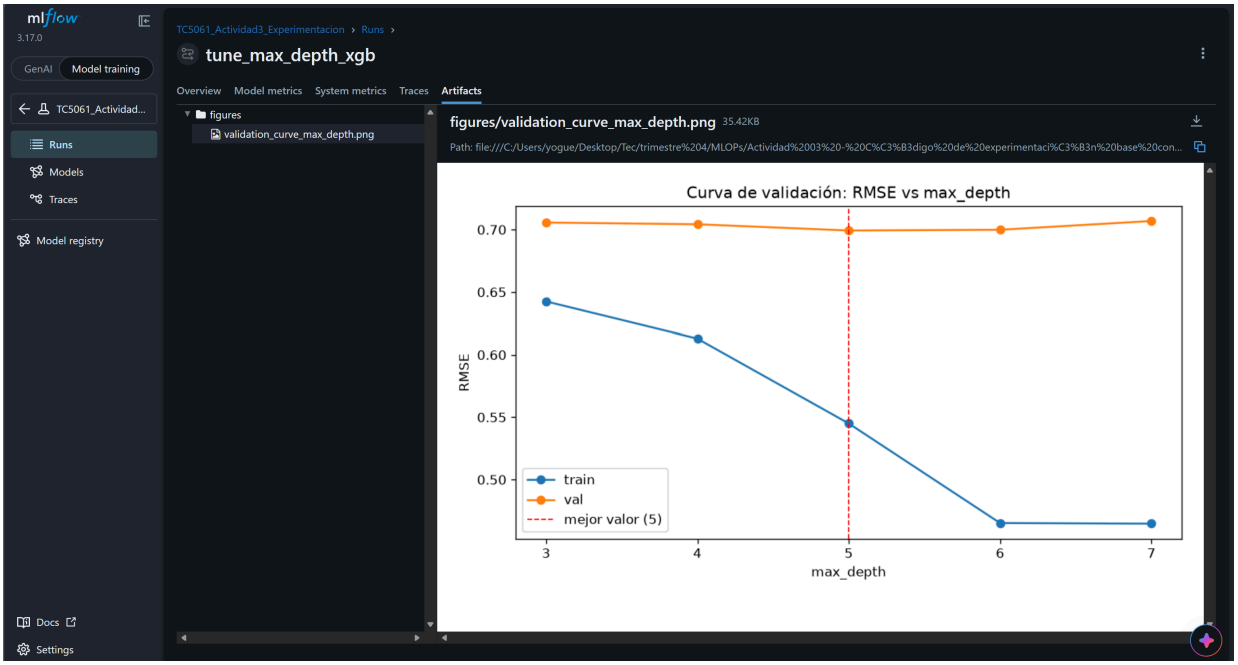
Captura 2 — Detalle de la corrida campeona `rf_ne300_md12_msl1_sin-norm_sin-esc`: las 16 métricas registradas, tags y metadatos de la corrida:



Captura 3 — Artefactos de la corrida campeona (carpeta `figures/` y `metrics_table.json` con las métricas por partición, incluida la referencia dummy):



Captura 4 — Curva de validación del sweep `tune_max_depth_xgb` (RMSE train vs val contra `max_depth`; mejor valor = 5):



4. Análisis de resultados

Mejor configuración. El campeón por validación fue **Random Forest con 300 árboles, profundidad máxima 12 y sin normalización ni escalado** (`rf_ne300_md12_msl1_sin-norm_sin-esc`): val RMSE **0.6935**, y en test —evaluado una sola vez, ya elegido— RMSE **0.6977**, MAE 0.538 y R^2 0.376, muy por encima de la referencia trivial de predecir la media (test RMSE 0.8831). Las tres variantes de random forest quedaron estadísticamente empatadas (val RMSE 0.6935–0.6939), de modo que la evidencia sostiene "random forest con ~200–400 árboles" más que una combinación exacta. El mejor XGBoost afinado (profundidad 5, lr 0.05, 165 árboles por early stopping) quedó cerca en validación (0.6993) — y de hecho en test rindió 0.6934, mejor que el campeón: una inversión dentro del ruido que confirma el empate práctico entre

ensambles (la selección se mantiene por validación para no contaminar test). El lineal (0.7410) y el árbol (0.7483) quedaron claramente atrás.

Hiperparámetro más influyente. La decisión que más movió las métricas fue la **familia de modelo**: pasar de lineal/árbol a un ensamble redujo el val RMSE ~0.05 (≈7%). Dentro de XGBoost, el hiperparámetro individual más influyente fue **max_depth**, con curva en U en el sweep (val RMSE 0.6993 en profundidad 5 frente a 0.7057/0.7069 en los extremos 3 y 7, y 0.7094 con profundidad 2 en la búsqueda manual), mientras `learning_rate` y `n_estimators` solo movieron milésimas una vez fijada la profundidad con early stopping. Hallazgo adicional: quitar la normalización y el escalado **no afectó** a los modelos de árboles (el campeón no los usa); ese preprocesamiento beneficia a los modelos lineales.

Sobreajuste del modelo seleccionado. El campeón **está sobreajustado**: su RMSE de entrenamiento (0.391) es 77% mejor que el de validación (0.6935), brecha que el script detecta y etiqueta (`overfit_gap_pct=77.4`, `is_overfitting=si`, umbral 3%); todos los ensambles del experimento presentan brechas de 67–172%, típicas de árboles que memorizan el conjunto de entrenamiento. La consistencia val→test del campeón (0.6935→0.6977, +0.6%) indica que aun así generaliza mejor que las alternativas de baja capacidad (el lineal, único modelo con brecha <3%, rinde peor en test: ~0.74). **Lo ideal sería iterar con el sweep (`tune`) sobre cada hiperparámetro de capacidad** (`max_depth`, `min_samples_leaf`, `n_estimators`, y en xgb regularización) **hasta una configuración con brecha train-val menor al 3% sin sacrificar validación**: un modelo que rinda igual sin memorizar es preferible por robustez ante cambios en la distribución de los datos.

Limitaciones. La principal es la **selección de hiperparámetros**: la búsqueda fue por coordenadas (un hiperparámetro a la vez), que puede perder interacciones entre ellos, y sobre una única partición (semilla 42). Además de la estrategia de sweeps con restricción de brecha descrita arriba, podrían escogerse mejor con búsqueda aleatoria o bayesiana (Optuna) para cubrir combinaciones, y con validación cruzada para medir la varianza entre particiones.

Desde la perspectiva de MLOps, el ejercicio también tiene límites claros:

- **Datos:** se descargan y colocan a mano y no están versionados (DVC los versionaría junto al código); no hay validación automática de esquema/calidad a la entrada del pipeline (tipo Great Expectations) — un CSV corrupto llegaría al entrenamiento sin aviso.
- **Selección y promoción del modelo:** la elección del campeón es manual (mirar la tabla); falta registrarlo en el **Model Registry** con promoción por alias (p. ej. `@champion`) y un *gate* automático de validación que compare el candidato contra el modelo vigente antes de promoverlo.
- **Evaluación estática:** test es un corte fijo del mismo dataset; en un ciclo real la evaluación se refresca con datos recientes y se complementa con monitoreo de *drift* y evaluación continua, porque la relación química–calidad puede cambiar entre cosechas.
- **Entorno y automatización:** las versiones están fijadas pero no hay contenedor (Docker) que garantice el entorno entre máquinas; el tracking vive en SQLite local (no compartible en equipo: haría falta un servidor de tracking remoto) y no existe CI/CD ni reentrenamiento automatizado.

Reproducibilidad: semilla fija y registrada por corrida; `requirements.txt` con versiones fijadas; README con instrucciones exactas de ejecución.

Declaración de uso de IA: Anthropic. (2026). *Claude Code (Claude Opus 5.5 y Claude Fable 5)* [Modelo de lenguaje grande], utilizado para edición de código y gestión del repositorio bajo un esquema *human-in-the-loop* (decisiones dirigidas, revisadas y aprobadas por el autor). https://claude.com/claude-code